

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

API 函数自搜索实验

实验要求：

复现第五章实验七，基于示例 5-11，完成 API 函数自搜索的实验，将生成的 exe 程序，复制到 windows 10 操作系统里验证是否成功。

实验过程：

• 本次实验将以教材中的案例为基础，需要设计一段通用的 shellcode 代码，确保其能在多种系统环境中正常运行。为此，shellcode 需具备动态搜索 API 函数地址的能力，从而实现跨平台的兼容性。再逐步演示如何实现 API 函数的动态地址搜索。最终，我们将通过弹窗显示"westwest"来验证这一机制的有效性。

1.复现第五章实验七，在关键位置设置断点，以便更直观地观察和分析 API 函数自搜索的具体实现机制。

• 在 mov esi, esp 指令之前，程序会执行 3 次 push 操作，分别压入 MessageBoxA、ExitProcess 和 LoadLibraryA 这三个 API 函数的预计算哈希值（这些哈希值由单独的工具程序提前计算得到）。后续在比对函数名称时，程序并非直接比较字符串，而是比较这些哈希值。

• 寄存器 esi 的值 0012FF28 在整个运行过程中保持不变，始终指向栈中存储的这三个哈希值。

```
CLD                                //清空标志位DF
push 0x1E380A6A                    //压入MessageBoxA的hash-->user32.dll
push 0x4FD18963                    //压入ExitProcess的hash-->kernel32.dll
push 0x0C917432                    //压入LoadLibraryA的hash-->kernel32.dll
mov esi, esp                        //esi=esp,指向堆栈中存放LoadLibraryA的hash
lea edi, [edi-0xc]                 //空出8字节应该也是为了兼容性
//=====开辟一些栈空间
xor ebx, ebx
mov bh, 0x04
sub esp, ebx                       //esp-=0x400
//=====压入"user32.dll"
mov bx, 0x3233
push ebx                           //0x3233
push 0x72657375                    //"user"
```

Registers	
EAX	CCCCCCCC
ECX	00000000
ESI	0012FF28
EIP	0040103A
EBP	0012FF80
DS	0023
ES	
GS	0000
OU	0
PE	0
CY	0
ST0	+0.000000

• mov bh, 0x04 设置 ebx 高位为 0x0400，随后 sub esp, ebx 上移栈指针，预留栈空间。

```
push 0x0C917432                    //压入LoadLibraryA的hash-->kernel32.dll
mov esi, esp                        //esi=esp,指向堆栈中存放LoadLibraryA的hash的地址
lea edi, [edi-0xc]                 //空出8字节应该也是为了兼容性
//=====开辟一些栈空间
xor ebx, ebx
mov bh, 0x04
sub esp, ebx                       //esp-=0x400
//=====压入"user32.dll"
mov bx, 0x3233
push ebx                           //0x3233
push 0x72657375                    //"user"
push esp                           //user
```

Registers	
EAX	00000000
ECX	00000000
EDX	00430DE0
ESI	0012FF28
EDI	0012FF10
EIP	00401041
ESP	0012FF28
EBP	0012FF80
EFL	00000246
CS	00
DS	0023
ES	0023
SS	0023
FS	00
GS	0000
OU	0
UP	0
EI	1
PL	0
ZR	1
AC	
PE	1
CY	0
ST0	0.00000000000000000000000000000000

•接着，程序通过两次 push 指令将字符串 user 和 32 压入栈中，组合成 user32.dll 以便动态加载该库

```
xor     eax,eax
mov     bh,0x04
sub     esp,ebx           //esp--0x400
//=====压入"user32.dll"
mov     bx,0x00003233
push    ebx               //0x3233
push    0x72657375        //"user"
push    esp
xor     edx,edx           //edx=0
//=====找kernel32.dll的基地址
mov     ebx,fs:[edx+0x30] //[[TEB+0x30]-->PEB
mov     ecx,[ebx+0xC]     //[PEB+0xC]-->PEB_LDR_DATA
mov     ecx,[ecx+0x1C]    //[PEB_LDR_DATA+0x1C]-->InInitializationOrderModuleList
mov     ecx,[ecx]         //进入链表第一个就是ntdll.dll
```

Registers

EAX	=	CCCCCCCC	EBX	=	00003233	
ECX	=	00000000	EDX	=	00430DE0	
ESI	=	0012FF28	EDI	=	0012FF1C	
EIP	=	00401048	ESP	=	0012FB24	
EBP	=	0012FF80	EFL	=	00000206	
DS	=	0023	ES	=	0023	
SS	=	0023	FS	=	0000	
GS	=	0000	OU=0	UP=0	EI=1	
PL=0	ZI	PE=1	CV=0	ST0	=	+0.0000000000000000e+000

• push 0x72657375 压入 user，push 0x00003233 压入 32 组合成"user32.dll"字符串，用于 LoadLibraryA 加载。

```
mov     esi,esp           //esi=esp,指向堆栈中存放LoadLibraryA的hash的地址
lea     edi,[esi-0xC]     //空出8字节应该也是为了兼容性
//=====开辟一些栈空间
xor     ebx,ebx
mov     bh,0x04
sub     esp,ebx           //esp--0x400
//=====压入"user32.dll"
mov     bx,0x00003233
push    ebx               //0x3233
push    0x72657375        //"user"
push    esp
xor     edx,edx           //edx=0
//=====找kernel32.dll的基地址
mov     ebx,fs:[edx+0x30] //[[TEB+0x30]-->PEB
mov     ecx,[ebx+0xC]     //[PEB+0xC]-->PEB_LDR_DATA
mov     ecx,[ecx+0x1C]    //[PEB_LDR_DATA+0x1C]-->InInitializationOrderModuleList
mov     ecx,[ecx]         //进入链表第一个就是ntdll.dll
```

Registers

EAX	=	CCCCCCCC	EBX	=	00003233	
ECX	=	00000000	EDX	=	00430DE0	
ESI	=	0012FF20	EDI	=	0012FF10	
EIP	=	0040104D	ESP	=	0012FB20	
EBP	=	0012FF80	EFL	=	00000206	
DS	=	0023	ES	=	0023	
SS	=	0023	FS	=	0000	
GS	=	0000	OU=0	UP=0	EI=1	
PL=0	ZI	PE=1	CV=0	ST0	=	+0.0000000000000000e+000

Address: 0x0012FB20

0012FB20	75 73 65 72 33 32 00 00 0C 0D 0E 0F 10 11 12 13	user32.....
0012FB30	14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23	!""#
0012FB40	24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33	\$%&'()*+,-./0123
0012FB50	34 35 36 37 38 39 3A 3B 3C 3D 3E 3F 40 41 42 43	456789:;<=>?@abc

main() line 22
mainCRTStartup()
KERNEL32! 7c81706

• push esp 将当前栈指针压入栈中。此时：esp 值减小（栈增长）、新栈顶 0x0012FB1C 存储的是前栈顶 0x0012FB20（指向"user32.dll"字符串）、形成调用约定要求的参数布局，可直接用于 LoadLibraryA 调用

注：push esp 实现了将字符串地址作为参数传递，同时保持栈结构正确）

```
sub     esp,ecx           //esp--0x700
//=====压入"user32.dll"
mov     bx,0x00003233
push    ebx               //0x3233
push    0x72657375        //"user"
push    esp
xor     edx,edx           //edx=0
//=====找kernel32.dll的基地址
mov     ebx,fs:[edx+0x30] //[[TEB+0x30]-->PEB
mov     ecx,[ebx+0xC]     //[PEB+0xC]-->PEB_LDR_DATA
mov     ecx,[ecx+0x1C]    //[PEB_LDR_DATA+0x1C]-->InInitializationOrderModuleList
mov     ecx,[ecx]         //进入链表第一个就是ntdll.dll
```

Registers

ESI	=	0012FF20	EPI	=	0012FF10	
EIP	=	0040104E	ESP	=	0012FB1C	
EBP	=	0012FF80	EFL	=	00000206	
DS	=	0023	ES	=	0023	
SS	=	0023	FS	=	0000	
GS	=	0000	OU=0	UP=0	EI=1	
PL=0	ZI	PE=1	CV=0	ST0	=	+0.0000000000000000e+000

Address: 0x0012FB1C

0012FB1C	20 FB 12 00 75 73 65 72 33 32 00 00 0C 0D 0E 0F	...user32.....
0012FB2C	10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F	

main() line 23
mainCRTStartup()
KERNEL32! 7c81706

•通过 fs:[0x30] 获取 PEB，遍历 PEB_LDR_DATA，最终找到 kernel32.dll 的基地址（如 0x7C800000）并存入 EBP 寄存器。

```
push    esp
xor     edx,edx           //edx=0
//=====找kernel32.dll的基地址
mov     ebx,fs:[edx+0x30] //[[TEB+0x30]-->PEB
mov     ecx,[ebx+0xC]     //[PEB+0xC]-->PEB_LDR_DATA
mov     ecx,[ecx+0x1C]    //[PEB_LDR_DATA+0x1C]-->InInitializationOrderModuleList
mov     ecx,[ecx]         //进入链表第一个就是ntdll.dll
mov     ebp,[ecx+0x8]     //ebp=kernel32.dll的基地址

//=====是否找到了自己所需全部的函数
Find_lib_functions:
lodsd   [esi]             //即move eax,[esi], esi+=4, 第一次取LoadLibraryA的hash
cmp     eax,0x1E380A6A     //与MessageBoxA的hash比较
jne     find_functions    //如果没有找到MessageBoxA函数，继续找
xchg    eax,ebp           //
call    [edi-0x8]         //LoadLibraryA("user32")
xchg    eax,ebp           //ebp=user32.dll的基地址,eax=MessageBoxA的hash <-- |
```

Registers

EAX	=	CCCCCCCC	EBX	=	00003233	
ECX	=	00242020	EDX	=	00430DE0	
ESI	=	0012FF28	EDI	=	0012FF1C	
EIP	=	0040105F	ESP	=	0012FB24	
EBP	=	7C800000	EFL	=	00000206	
DS	=	0023	ES	=	0023	
SS	=	0023	FS	=	0000	
GS	=	0000	OU=0	UP=0	EI=1	
PL=0	ZI	PE=1	CV=0	ST0	=	+0.0000000000000000e+000

• lodsd 指令自动从 esi 指向的地址加载 4 字节数据到 eax 并递增 esi，用于遍历预先压入栈中的 API 函数哈希值（如 0x0C917432 对应 LoadLibraryA 的哈希），通过比对哈希值而非函数名来动态解析 API 地址，既提高了 shellcode 的隐蔽性，又确保了跨系统兼容性。

```

mov     ecx,[ecx+0x1C]    // [PEB_LDR_DATA+0x1C]--->InInitializ
mov     ecx,[ecx]         // 进入链表第一个就是ntdll.dll
mov     ebp,[ecx+0x8]     // ebp= kernel132.dll的基地址

//=====是否找到了自己所需全部的函数
find_lib_functions:
lodsd   //即move eax,[esi], esi+=4, 第一次取LoadLibraryA的hash
cmp     eax,0x1E380A6A    //与MessageBoxA的hash比较
jne     find_functions    //如果没有找到MessageBoxA函数, 继续找
xchg    eax,ebp           //-----
call    [edi+0x8]         //LoadLibraryA("user32")
xchg    eax,ebp           //ebp=kernel132.dll的基地址,eax=MessageBoxA的hash <-- |

//=====导出函数名列表指针
find_functions:

```

Registers			
EAX	= 0C917432	EBX	= 7F
ECX	= 00242020	EDX	= 00
ESI	= 0012FF2C	EDI	= 00
EIP	= 00401060	ESP	= 00
EBP	= 7C800000	EFL	= 00
DS	= 0023	ES	= 0023
GS	= 0000	OU	= 0
UP	= 0	EI	=
PE	= 1	CV	= 0
ST0	= +0.0000000000000000		

• 通过循环比对哈希值来动态解析 API 函数地址：首先用 cmp 判断当前哈希是否为最后一个，若不是则通过 jne 跳转继续查找；在循环中，程序会遍历目标 DLL 的 PE 导出表，计算每个函数名的哈希值并与目标哈希（如 LoadLibraryA 的 0x0C917432）进行比对，匹配后会将函数地址存入 ebp；最终把找到的函数地址保存到 edi 中。

```

next_function_loop:
inc     edi
mov     esi,[ebx+edi*4]    //从列表数组中读取
add     esi,ebp           //esi = 函数名称所在地址
cdq

//=====函数名的hash运算
hash_loop:
movsx   eax,byte ptr[esi]
cmp     al,ah             //字符串结尾就跳出当前函数
jz      compare_hash
ror     edx,7
add     edx,eax
inc     esi
jnp     hash_loop
//=====比较找到的当前函数的hash是否是自己想找的
compare_hash:
cmp     edx,[esp+0x1C]    //lods pushad后,栈+1c为LoadLibraryA的hash

compare_nash:
cmp     edx,[esp+0x1C]    //lods pushad后,栈+1c为LoadLibraryA的hash
jnz     next_function_loop
mov     ebx,[ecx+0x24]    //ebx = 顺序表的相对偏移量
add     ebx,ebp           //顺序表的基地址
mov     di,[ebx+2*edi]    //匹配函数的序号
mov     ebx,[ecx+0x1C]    //地址表的相对偏移量
add     ebx,ebp           //地址表的基地址
add     ebp,[ebx+4*edi]   //函数的基地址
xchg    eax,ebp          //eax<=>ebp 交换

pop     edi
stosd   //把找到的函数保存到edi的位置
push    edi

popad
cmp     eax,0x1E380A6A    //找到最后一个函数MessageBox后, 跳出循环
jne     find_lib_functions //如果不是继续循环

```

Registers			
EAX	= 00000041		
ECX	= 7C80262C	EDX	= 0C917432
ESI	= 7C804BAA		
EIP	= 00401087		
EBP	= 7C800000		
DS	= 0023	ES	=
GS	= 0000	OU	= 0
PE	= 1	CV	= 0
ST0	= +0.000000		

Registers			
EAX	= 00000000	EBX	= 7C80262C
ECX	= 7C80262C	EDX	= 0C917432
ESI	= 7C807649	EDI	= 00000020
EIP	= 004010AA	ESP	= 0012FA10
EBP	= 7C801D7B	EFL	= 00000020
DS	= 0023	ES	= 0023
GS	= 0000	OU	= 0
PE	= 1	CV	= 0
ST0	= +0.0000000000000000e-		

```

Address: 0x0012FB1C
0012FB1C  20 FB 12 00 75 73 65 72 33 32 00 00 0C 0D 0E 0F  ...user32.....
0012FB2C  10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F  .....
0012FB3C  20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F  !'#$%&'()*+,-./

```

• 完成动态 API 解析后，当最后一个目标函数匹配成功时退出循环，此时 esp 指向栈中预置的 user32.dll 字符串地址(0x0012FB20)，通过 call edi 指令调用已找到的 LoadLibraryA 函数加载该动态库，为后续 API 调用做好准备。


```

find_lib_functions:
    lodsd      //即move eax,[esi], esi+=4, 第一次取LoadLibraryA的hash
    cmp       eax,0x1E380A6A //与MessageBoxA的hash比较
    jne       find_functions //如果没有找到MessageBoxA函数, 继续找
    xchg      eax,ebp
    call      [edi-0x8] //LoadLibraryA("user32")
    xchg      eax,ebp //ebp=user32.dll的基地址,eax=MessageBoxA的hash

    //=====导出函数名列表指针
find_functions:
    pushad
    mov       eax,[ebp+0x3C] //保护寄存器
    mov       ecx,[ebp+eax+0x78] //dll的PE头
    add       ecx,ebp //导出表的指针
    mov       ebx,[ecx+0x20] //导出函数名列表指针
    //=====找下一个函数名
next_function_loop:

```

Registers

EAX	= 77D10000	EBX	= 7FFD5000
ECX	= 7C801BFA	EDX	= 00140608
ESI	= 0012FF34	EDI	= 0012FF24
EIP	= 0040106B	ESP	= 0012FB20
EBP	= 1E380A6A	EFL	= 00000246
CS	= 0023	ES	= 0023
SS	= 0023	FS	= 0023
GS	= 0000	OU	= 0
UP	= 0	EI	= 1
PL	= 0	ZR	= 1
PE	= 1	CY	= 0
ST0	= +0.0000000000000000e+0000		

• 动态加载获取 user32.dll 的基地址 0x77D10000 并更新至 ebp 寄存器，整个过程完全基于内存遍历和哈希校验，无需任何硬编码，确保 shellcode 跨 Windows 版本通用性。

```

    cmp       eax,0x1E380A6A //与MessageBoxA的hash比较
    jne       find_functions //如果没有找到MessageBoxA函数, 继续找
    xchg      eax,ebp
    call      [edi-0x8] //LoadLibraryA("user32")
    xchg      eax,ebp //ebp=user32.dll的基地址,eax=MessageBoxA的hash

    //=====导出函数名列表指针
find_functions:
    pushad
    mov       eax,[ebp+0x3C] //保护寄存器
    mov       ecx,[ebp+eax+0x78] //dll的PE头
    add       ecx,ebp //导出表的指针
    mov       ebx,[ecx+0x20] //导出函数名列表指针
    add       ebx,ebp //导出函数名列表指针的基地址
    xor       edi,edi

    //=====找下一个函数名
next_function_loop:

```

Registers

EAX	= 1E380A6A	EBX	= 7FFD5000
ECX	= 7C801BFA	EDX	= 00140608
ESI	= 0012FF34	EDI	= 0012FF24
EIP	= 0040106C	ESP	= 0012FB20
EBP	= 77D10000	EFL	= 00000246
CS	= 0023	ES	= 0023
SS	= 0023	FS	= 003B
GS	= 0000	OU	= 0
UP	= 0	EI	= 1
PL	= 0	ZR	= 1
AC	= 0	PE	= 1
CY	= 0	ST0	= +0.0000000000000000e+0000

• 构造 MessageBoxA 的调用参数，通过已解析的函数地址触发弹窗显示 westwest，随后调用 ExitProcess 安全终止进程，期间使用 NOP 指令保持内存对齐，确保 shellcode 在不同内存偏移环境下都能稳定执行。

```

function_call:
    xor       ebx,ebx
    push     ebx
    push     0x74736577 //push "westwest"
    mov      eax,esp
    push     ebx
    push     eax
    push     ebx
    call     [edi-0x04] //MessageBoxA(NULL,"westwest")
    push     ebx
    call     [edi-0x08] //ExitProcess(0);
    nop
    nop
    nop
    nop
    }
    return 0;

```

• 在 VC6 中运行显示 westwest 对话框：

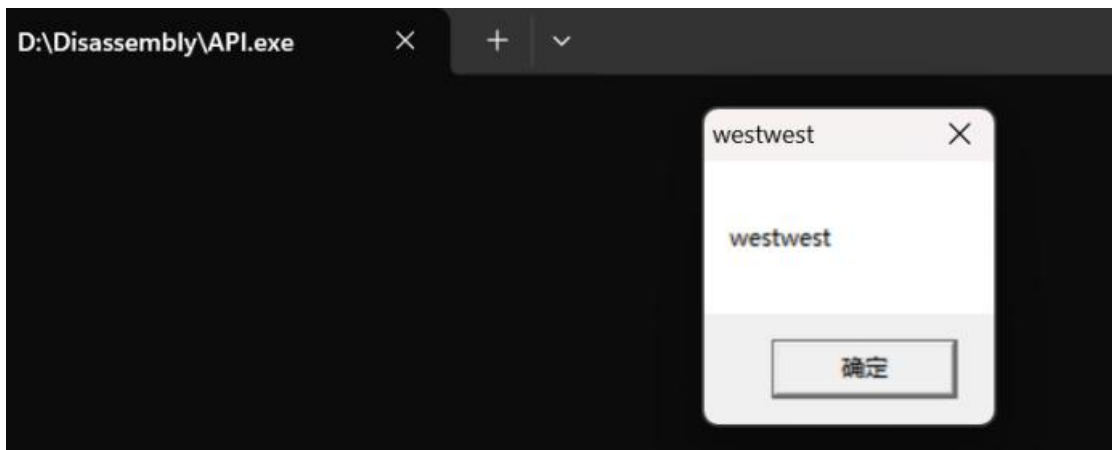
```

push 0x1E380A6A
push 0x4FD18963
push 0x0C917432
mov esi,esp //e
lea edi,[esi-0xc]
//=====开辟一些栈空间
xor ebx,ebx
mov bh,0x04
sub esp,ebx
//=====压入"user32.dll"
mov bx,0x3233
push ebx
push 0x72657375
push esp
xor edx,edx
//=====找kernel32.dll的基地址
mov ebx,fs:[edx+0x30]
mov ecx,[ebx+0xC]

```



2. 将生成的 exe 程序，复制到 windows 10 操作系统里验证成功。



心得体会：

1. 通过 PEB 遍历和哈希比对实现 API 定位，避免了硬编码依赖，使 Shellcode 适配不同 Windows 版本。关键技巧如 push esp 传参、NOP 指令对齐，兼顾了功能与稳定性。
2. 断点分析寄存器状态（如 ebp 基址切换）验证了动态加载流程，解决哈希比对问题时，认识到算法一致性的重要性。
3. 实验将 PE 结构、导出表等理论具象化，如 MessageBoxA 地址的动态解析与调用。现代防护技术（如 ASLR）对 Shellcode 的影响值得进一步研究。